Project Scalability & Reliability Assessment

System: AI-Native Organization OS

Assessment date: May 24, 2026

Version assessed: Current main branch (Postgres-first, single-process API)

Prepared for: Platform operations and engineering leadership

Executive summary

The AI-Native Organization OS is a production-capable single-instance coordination platform with strong data consistency primitives (transactional writes, idempotent events, per-project orchestration locks) and operational reliability (readiness gates, graceful shutdown, structured errors, health checks). It is not yet horizontally scalable without externalizing in-memory state (event log cache, LLM queue, SSE fan-out, orchestration locks).

Dimension	Rating (1–5)	Summary
Reliability	4 / 5	Postgres source of truth, health/shutdown…
Consistency	4 / 5	Transactional persist; per-project…
Scalability (vertical)	3 / 5	In-memory event log + SSE cap; pool…
Scalability (horizontal)	2 / 5	Requires Redis/distributed queue before…
Observability	4 / 5	Ops Monitor, agent activity, LLM logs,…
Operational readiness	4 / 5	Documented env vars, readiness probe,…

Recommendation: Run one API process per environment today. Plan Redis + worker tier before scaling API replicas or heavy concurrent LLM load.

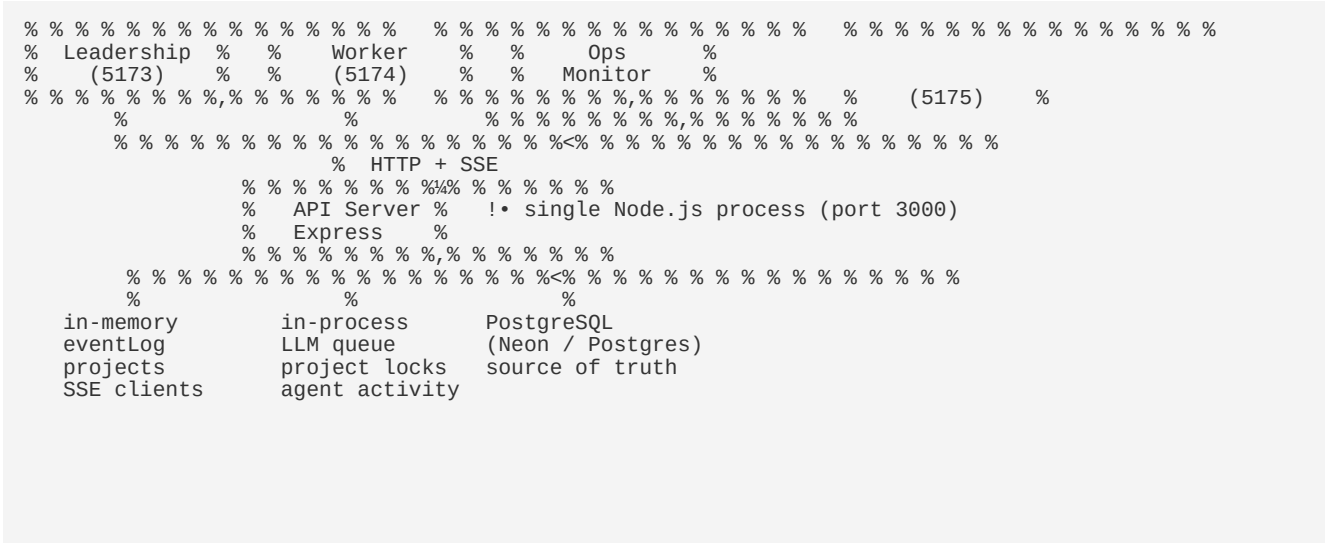
---

1. System context

1.1 Purpose

The platform automates corporate coordination (planning, assignment, scheduling, project monitoring, worker requests) while keeping humans in the loop for judgment and execution. Events drive project state; AI agents react to signals and delegate work.

1.2 Architecture (current)



1.3 Technology stack

Layer	Technology
API	Node.js 18+, Express
Database	PostgreSQL (`pg` pool, Neon-compatible)
Real-time	Server-Sent Events (SSE), capped clients
AI	Gemini, OpenAI, DeepSeek, Ollama (optional)
Frontends	React 18 + Vite (3 separate apps)

---

2. Reliability assessment

2.1 Strengths

Capability	Implementation	Impact
Persistence before listen	`initStore()` completes before HTTP…	Avoids serving stale or empty state
Readiness / liveness	`GET /api/health` — `SELECT 1`,…	Load balancers can drain unhealthy…
Graceful shutdown	`SIGTERM` / `SIGINT` !' hooks !' close HTT…	Clean deploys; in-flight work bounded (15…
503 during startup/shutdown	Middleware rejects non-health routes until…	Prevents partial writes during boot
Transactional writes	`persistEventAndState()` — event + proje…	Reduces split-brain between event and…
Idempotent events	`eventExistsById` + `ON CONFLICT DO…	Safe retries from clients and agents
Standard errors	`apiErrors.js` — `{ error: { code, message…	Predictable client handling
SSE backpressure	`SSE_MAX_CLIENTS` (default 200)	Limits connection exhaustion
Pool resilience	Idle client error handler on `pg` pool	Survives transient Neon/Postgres…
AI handler re-queue	Re-schedules when prior run still…	Avoids dropped leadership automation
Staggered Project AI	3 projects per poll tick	Reduces LLM/API thundering herd

2.2 Risks and gaps

Risk	Severity	Notes
Single process = single point of failure	High	No active-active failover without…

Full event log in memory	Medium	Startup load time and RAM grow with eve…
In-process LLM queue	Medium	One global lock; long calls block other…
No automated backup/runbook in repo	Medium	Depends on Neon/hosting provider RPO/…
No rate limiting on public routes	Low–Medium	Abuse or accidental loops could stress D…
Agent activity hydrate at startup	Low	Monitor history may be sparse until new…

2.3 Reliability scorecard

Criteria	Met?	Evidence
Database is source of truth	Yes	All durable state via `postgresStore`
Fail closed on DB down	Yes	Health returns 503; startup fails if init fails
Duplicate events rejected	Yes	Memory + DB idempotency check
Orchestration races per project	Mitigated	`runWithProjectLock(projectId)`
Deploy without corrupting state	Yes	Graceful shutdown + transactional persist

---

3. Scalability assessment

3.1 Current scale envelope (estimated)

Resource	Typical dev/staging	Bottleneck
Events in DB	2,000–2,500+	Memory replay on cold start
Projects	7–10	Project AI poll fan-out
People	~20	Workforce analytics queries
Worker needs	300+	Leadership AI Handler batch (8/run)
SSE clients	"d 200 (configurable)	Per-process connection limit
Postgres pool	2–20 (`PG_POOL_MAX`)	Concurrent route handlers

Suitable today for: single org pilot, tens of projects, hundreds of events/hour, small leadership + worker cohort.

Not suitable without changes for: multiple API replicas, thousands of events/minute, hundreds of simultaneous SSE subscribers across instances.

3.2 Vertical scaling (single instance)

Lever	Status
Increase `PG_POOL_MAX`	Supported (cap 20)
Neon pooler connection string	Recommended in production
`OPS_MONITOR_STREAM_HOURS`	Tunable monitor window
Staggered Project AI polling	Implemented
Replan dedup (15 min window)	Reduces redundant LLM work

Limit: Node.js single-threaded event loop + in-memory `eventLog` size + global LLM mutex.

3.3 Horizontal scaling (not implemented)

To run N API instances, the following must be externalized:

Component	Today	Required for scale-out
Orchestration locks	In-memory `Map` per process	Redis or Postgres advisory locks
LLM queue	In-process lock + queue	Redis queue or dedicated worker service
SSE broadcasts	In-memory client set	Redis pub/sub or SSE gateway
Event log reads	Full load into RAM	Time-window queries or read replicas
Session / cache (future)	N/A	Redis optional

Documented roadmap: `docs/reliability.md` § Horizontal scale (future).

3.4 Scalability scorecard

Criteria	Current	Target (scale-out)
Stateless API	No	Yes, with external locks/queue
Shared pub/sub	No	Redis pub/sub
DB connection pooling	Yes	Neon pooler + right-sized pool per instance
Background job tier	Partial (timers in API)	Dedicated workers for LLM/orchestration

---

4. Consistency & data integrity

4.1 Write path

1. Client posts event !' idempotency check (memory + DB).
2. State mutation in memory !' `persistEventAndState(event, projectState, needRecord)`.
3. Transaction commits event row + project snapshot (+ need if applicable).
4. SSE broadcast to connected clients.

4.2 Concurrency controls

Mechanism	Scope
`runWithProjectLock`	One orchestration/replan chain per `projectId`
`recentlyReplanned`	15-minute dedup window for Project AI replans
Leadership AI Handler debounce	1.2s debounce; re-queue if `processing`
LLM global queue	Single active model call at a time (per process)

4.3 Known consistency caveats

• Memory vs DB: In-memory cache is authoritative for reads during runtime; restart reloads from Postgres snapshot + incremental replay.

• Multi-instance: Without distributed locks, two processes could process the same project concurrently.

• Event ordering: Per-project lock serializes orchestration; global event order is append-only in DB.

---

5. Observability & operations

5.1 Monitoring surfaces

Surface	Audience	Data
Ops Monitor UI (`/monitor`, port 5175)	Operations	Agent streams, LLM queue, work boards
`GET /api/ops/monitor`	Automation	JSON agent status + stream segments
`GET /api/health`	Orchestration / K8s	DB ping, `storeReady`, uptime
Agent activity table	Postgres	Structured lines for stream history
LLM logs table	Postgres	Prompts/responses per project

5.2 Operational procedures

```
# Readiness (503 if DB down or still starting)
curl -s http://localhost:3000/api/health

# Graceful stop
kill -SIGTERM <pid>
```

Post-deploy checklist: Verify `status: "ok"` and `database: "up"` before traffic.

5.3 Environment variables (reliability-related)

Variable	Default	Purpose
`DATABASE_URL`	required	Postgres connection
`PG_POOL_MAX`	10	Connection pool size (max 20)
`PG_CONNECTION_TIMEOUT_MS`	10000	Connect timeout
`SSE_MAX_CLIENTS`	200	SSE connection cap
`OPS_MONITOR_STREAM_HOURS`	3	Monitor time window

---

6. Frontend & deployment scalability

App	Deploy model	Scale notes
Leadership (`client/`)	Static CDN / Vite build	Horizontally unlimited (stateless)
Worker (`worker/`)	Static CDN	Same
Monitor (`monitor/`)	Static CDN	Polls + SSE via API
API (`server/`)	Single Node process	Scale bottleneck

Frontends scale independently; API is the constraint.

---

7. Risk register (prioritized)

ID	Risk	Likelihood	Impact	Mitigation
R1	API process crash	Medium	High	Health checks, fast…
R2	LLM provider outage	Medium	Medium	Stub mode; queue…
R3	Event log memory growth	Medium	Medium	Archive strategy;…
R4	Duplicate orchestration…	Low today / High if scaled	High	Do not run multiple…

R5	SSE connection storm	Low	Medium	`SSE_MAX_CLIENTS`…
R6	Long-running LLM…	Medium	Medium	External job queue;…

---

8. Recommendations

8.1 Immediate (0–4 weeks)

1. Run one API instance per environment; document in runbooks.
2. Use Neon pooler URL in production (`-pooler` host).
3. Wire health checks to load balancer / platform (503 until `storeReady`).
4. Monitor via Ops Monitor + periodic `GET /api/health`.
5. Verify idempotency on critical event producers (clients, webhooks).

8.2 Near-term (1–3 months)

1. Introduce Redis for: distributed orchestration locks, LLM job queue, SSE pub/sub.
2. Background worker process for LLM and heavy Project AI batches.
3. Event archival or time-window hydration to cap memory on startup.
4. Rate limiting on `POST /events` and LLM-heavy routes.
5. Backup / restore drill with Neon PITR documentation.

8.3 Long-term (3–6 months)

1. Multi-instance API behind load balancer with shared Redis.
2. Read replicas or CQRS for analytics/workforce if query load grows.
3. SLOs: API p99 latency, event ingest success rate, LLM queue wait time.
4. Chaos testing: DB failover, SIGTERM during orchestration.

---

9. Conclusion

The AI-Native Organization OS demonstrates mature reliability patterns for a single-node coordination service: transactional persistence, idempotency, health-gated startup, graceful shutdown, and strong operational visibility through the Ops Monitor. Scalability is intentionally bounded to one API process until distributed locks and an external LLM queue (Redis-backed) are introduced.

Overall assessment: Reliable for pilot and single-tenant production at current event/project scale; scale-out ready in design with a clear documented path in `docs/reliability.md`.

Appendix A — Key source files

Area	Path
Reliability doc	`docs/reliability.md`
Lifecycle / health	`server/lib/platformLifecycle.js`
Project locks	`server/lib/projectLock.js`

API entry	`server/index.js`
Event persist	`server/store/postgresStore.js`
Ops monitor	`server/services/opsMonitor.js`
LLM queue	`server/lib/llm.js`

Appendix B — Assessment methodology

This assessment is based on:

• Static review of server architecture and reliability modules

• Documented platform behavior in `docs/reliability.md` and README

• Operational patterns observed during development (health API, Ops Monitor, Postgres counts)

• Industry standards for single-instance Node + Postgres services

This document is informational and does not constitute a formal third-party audit.

AI-Native Organization OS — Scalability & Reliability Assessment — May 2026